

BIG DATA ANALYTICS DASHBOARD (HADOOP + DJANGO)

A CASE STUDY REPORT

Submitted by

**BASITH NISTAR (RA2411003011771)
AKHILAN VTM (RA2411003011821)
SREE VARSHAN S (RA2411003011827)**



for the course

21CSC302J – COMPUTER NETWORKS

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

**DEPARTMENT OF DATA SCIENCE AND BUSINESS SYSTEMS
SCHOOL OF COMPUTING
FACULTY OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR - 603 203.**

Abstract

This report details the design and implementation of a Big Data Analytics Dashboard, leveraging Apache Hadoop for distributed data processing and storage, and Django for a web-based user interface. The project focuses on integrating these technologies to provide a scalable and efficient solution for analyzing large datasets. Key aspects covered include the network architecture required for seamless communication between Django and Hadoop components, security measures to protect sensitive data, and monitoring strategies to ensure optimal system performance. The report introduces a practical IoT data ingestion layer using Arduino microcontrollers and Raspberry Pi edge gateways, forming a complete end-to-end pipeline from physical sensor data collection through distributed Hadoop processing to real-time Django dashboard visualization. Real-world applications across healthcare, finance, smart cities, and telecommunications are also discussed. The report outlines the testing and validation processes undertaken to confirm the functionality and reliability of the integrated system.

Table of Contents

Abstract

1. Introduction

1.1 Background

1.2 Objectives

2. Network Design

2.1 Topology

2.2 Components

2.3 IP Addressing Scheme

2.4 IoT Data Ingestion Layer (Arduino & Raspberry Pi)

3. Routing Configuration

3.1 Router Configuration

3.2 Static and Dynamic Routing

4. Switching Configuration

4.1 Switch Configuration

4.2 VLANs

5. Inter-VLAN Routing

5.1 Layer 3 switching using SVIs

5.2 Subnetting

6. Security Measures

6.1 Access Control Lists (ACLs)

6.2 NAT and PAT

6.3 Port Security

7. Quality of Service (QoS)

7.1 QoS Configuration

8. Monitoring and Management

8.1 SNMP Configuration

8.2 Logging and Alerts

9. Testing and Validation

9.1 Simulation

9.2 Troubleshooting

10. Results and Evaluation

10.1 Performance Metrics

11. Real-World Applications

12. Conclusion

12.1 Summary

12.2 Lessons Learned

13. References

14. Appendices

TABLE OF FIGURES

FIGURE 1: TOPOLOGY OF FULL NETWORK

FIGURE 2: THREE-TIER APPLICATION ARCHITECTURE

FIGURE 3: HADOOP CLUSTER INTERNAL ARCHITECTURE

FIGURE 4: IOT DATA INGESTION PIPELINE

FIGURE 5: VLAN SEGMENTATION DIAGRAM

FIGURE 6: ACL TRAFFIC FILTERING FLOW

FIGURE 7: NAT/PAT ADDRESS TRANSLATION

FIGURE 8: QOS PRIORITY QUEUE DIAGRAM

FIGURE 9: SNMP MONITORING ARCHITECTURE

FIGURE 10: TRACEROUTE SUCCESSFUL

FIGURE 11: DHCP IP ALLOCATION

FIGURE 12: HIVE QUERY LATENCY GRAPH

FIGURE 13: HDFS THROUGHPUT CHART

FIGURE 14: DJANGO RESPONSE TIME VS. CONCURRENT USERS

FIGURE 15: PERFORMANCE MEASURE THROUGH PING TIME

1. Introduction

1.1 Background

In the era of big data, organizations face the challenge of processing, storing, and analyzing vast volumes of information to derive meaningful insights. Traditional relational database systems often struggle with the scale and velocity of such data. Apache Hadoop, an open-source framework, provides a robust solution for distributed storage and processing of large datasets across clusters of computers. Concurrently, web frameworks like Django offer powerful tools for building interactive and scalable web applications. This case study explores the integration of Hadoop's big data capabilities with Django's web development prowess to create a Big Data Analytics Dashboard. This dashboard aims to provide users with an intuitive interface to interact with and visualize data stored and processed within a Hadoop ecosystem, thereby bridging the gap between raw big data and actionable business intelligence. To ground this system in real-world applicability, the architecture is extended with an IoT data ingestion layer using Arduino microcontrollers and Raspberry Pi edge gateways, enabling live sensor data to feed directly into the Hadoop pipeline.

1.2 Objectives

The primary objective of this project is to design and implement a Big Data Analytics Dashboard that effectively leverages Apache Hadoop for backend data processing and Django for the front-end web interface. Specific objectives include: To establish a scalable network architecture that facilitates seamless communication between Django web servers and Hadoop cluster components (e.g., NameNode, DataNodes, YARN ResourceManager, HiveServer2). To implement robust security measures, including authentication and authorization mechanisms, to protect sensitive data within the Hadoop ecosystem and control access to the Django dashboard. To integrate Django with Hadoop components, specifically using PyHive for querying data stored in Apache Hive and potentially interacting with HDFS for file management. To develop a user-friendly web interface using Django that allows for data visualization, query execution, and basic management of Hadoop-related tasks. To design and implement an IoT data ingestion layer using Arduino sensors and Raspberry Pi edge gateways that stream real sensor data into HDFS. To define and configure network routing, switching, and IP addressing schemes suitable for a distributed big data and IoT environment. To establish monitoring and logging mechanisms to track the health, performance, and security of both the Hadoop cluster and the Django application. To conduct thorough testing and validation of the integrated system to ensure functionality, performance, and reliability under various load conditions.

2. Network Design

2.1 Topology

The network topology for the Big Data Analytics Dashboard project is designed to support a distributed Hadoop cluster, a Django web application, and an IoT sensor ingestion layer, ensuring high availability, scalability, and efficient data flow. A typical setup involves a multi-tier architecture:

Hadoop Cluster Tier: This tier comprises multiple nodes, including a NameNode (master for HDFS), DataNodes (store actual data), ResourceManager (master for YARN), NodeManagers (manage resources on data nodes), and potentially HiveServer2 for SQL-like querying. These nodes are interconnected via a high-speed local area network (LAN).

Application Tier: This tier hosts the Django web application servers. These servers communicate with the Hadoop cluster to fetch data and execute queries. For redundancy and load balancing, multiple Django application servers can be deployed.

IoT Edge Tier: This tier hosts Raspberry Pi gateways that collect sensor readings from Arduino nodes and stream the data to HDFS via the WebHDFS REST API over a dedicated IoT VLAN.

Client Tier: End-users access the Django dashboard from this tier via web browsers.

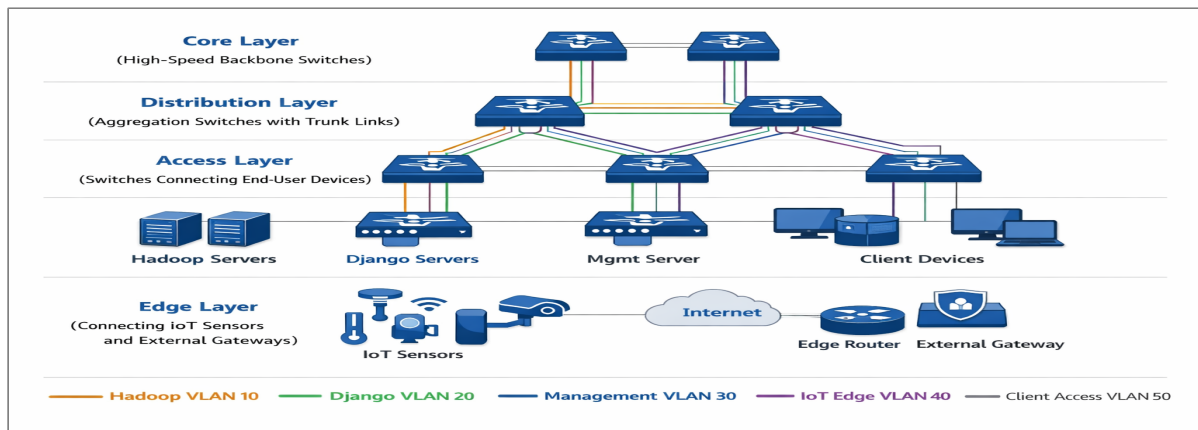


Figure 1: Topology of full network

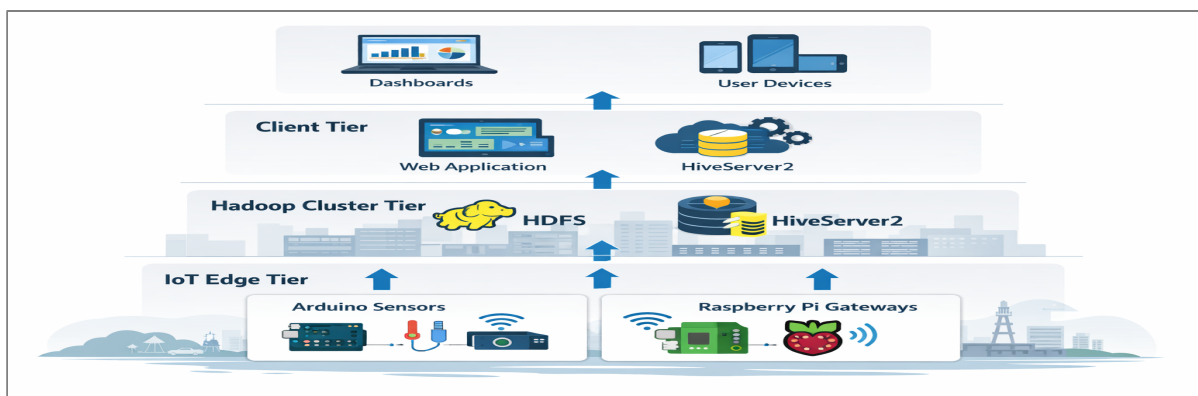


Figure 2: Four-Tier Application Architecture

2.2 Components

The network design incorporates the following key components:

- 1. Routers:** Act as gateways between different network segments (VLANs) and connect the entire system to external networks (e.g., the internet for client access). They handle routing decisions and enforce security policies.
- 2. Layer 3 Switches:** Provide inter-VLAN routing capabilities and high-speed connectivity within the data center. They are crucial for segmenting the network and optimizing traffic flow.
- 3. Layer 2 Switches:** Connect individual servers (Hadoop nodes, Django servers) within their respective VLANs. They provide port-level security and efficient local traffic management.
- 4. Hadoop Cluster Nodes:** NameNode: Manages the HDFS filesystem namespace and regulates access to files by clients. For high availability, a secondary NameNode or HA setup with multiple NameNodes is recommended. DataNodes: Store data in HDFS and report block information to the NameNode. ResourceManager: Manages the allocation of resources (CPU, memory) to applications running on YARN. NodeManagers: Run on each data node and manage resources and monitor containers. HiveServer2: Provides a Thrift-based server that enables clients to execute Hive queries over JDBC/ODBC, which Django will connect to via PyHive.
- 5. Django Application Servers:** Host the Django web application, serving the user interface and handling requests from clients. They interact with HiveServer2 to retrieve analytical data.
- 6. Raspberry Pi Edge Gateways:** Receive sensor readings from Arduino nodes over USB serial or WiFi and upload batched CSV data to HDFS using the WebHDFS REST API. They operate on a dedicated IoT VLAN (VLAN 40).
- 7. Load Balancers:** Distribute incoming client requests across multiple Django application servers to ensure high availability and optimal performance.
- 8. Firewalls:** Implement network security policies, controlling inbound and outbound traffic between different network zones and the internet.

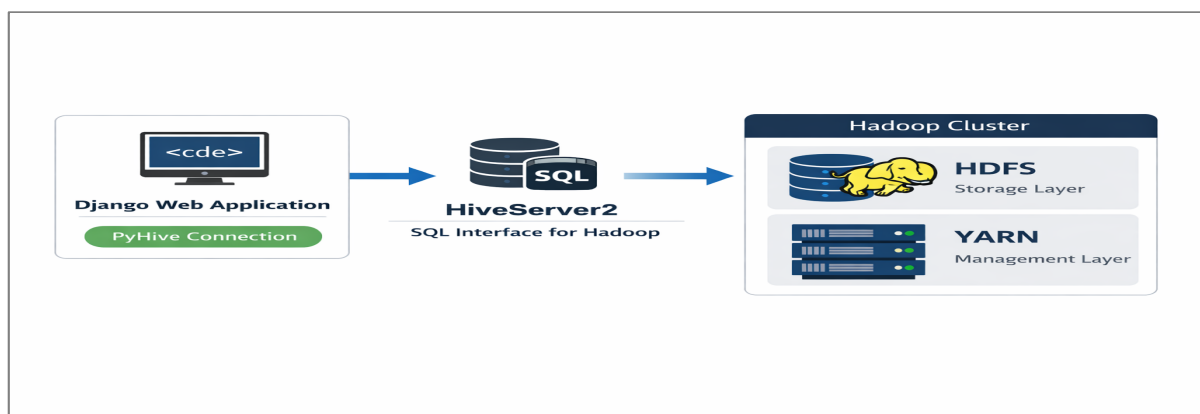


Figure 3: Hadoop Cluster Internal Architecture

2.3 IP Addressing Scheme:

A well-structured IP addressing scheme is critical for managing a complex network involving a Hadoop cluster, a Django application, and IoT edge devices. The scheme utilises private IP address ranges and is segmented using VLANs. Base Network: 10.0.0.0/16.

VLAN	Name	Subnet	Purpose
VLAN 10	Hadoop Cluster	10.0.10.0/24	NameNode, DataNodes, YARN, HiveServer2
VLAN 20	Django Application	10.0.20.0/24	Django web servers, load balancers
VLAN 30	Management	10.0.30.0/24	Admin workstations, SNMP server, logging
VLAN 40	IoT Edge	10.0.40.0/24	Raspberry Pi gateways, Arduino nodes
VLAN 50	Client Access	10.0.50.0/24	End-user devices, external access

Static IP Assignments: Critical components like NameNodes, ResourceManager, HiveServer2, Django application servers, load balancers, and Raspberry Pi gateways will be assigned static IP addresses for consistent network identification and service discovery.

Dynamic IP Assignments (DHCP): While not typically used for core cluster components, DHCP could be used for temporary nodes or management workstations within specific VLANs.

2.4 IoT Data Ingestion Layer (Arduino & Raspberry Pi)

The sections above define the network for processing and serving data. This section introduces the physical data source layer: Arduino microcontrollers collect real-world sensor data and transmit it to Raspberry Pi edge gateways, which upload the data to HDFS, completing the end-to-end pipeline from the physical world into the Hadoop cluster.

Hardware Roles

Platform	Role in the Project	Connection
Arduino Uno / ESP32	Reads physical sensors (temperature, humidity, CO2, motion). Transmits readings every 5 seconds over USB serial or WiFi.	USB serial or WiFi (ESP32) to Raspberry Pi
Raspberry Pi 3B+/4	Edge gateway: collects Arduino data, timestamps and formats it, then uploads batched CSV files to HDFS via WebHDFS REST API on port 50070.	Ethernet on VLAN 40 to Hadoop VLAN 10
Raspberry Pi 4 (optional)	Lightweight Hadoop DataNode for small-scale or lab/demo deployments. Runs <code>hadoop-daemon.sh start datanode</code> .	VLAN 10 - part of the Hadoop cluster

End-to-End Pipeline

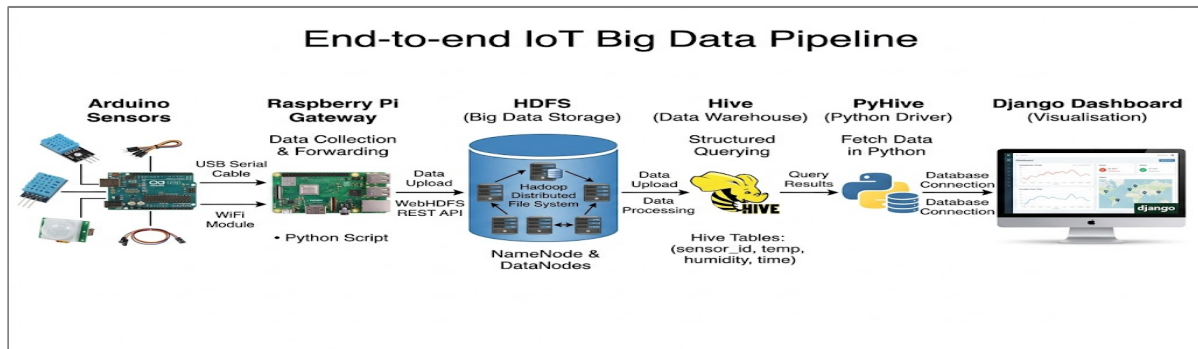


Figure 4: IoT Data Ingestion Pipeline Architecture

Stage 1 -- Sensor Data Collection (Arduino)

Arduino microcontrollers equipped with sensors continuously sample physical measurements. Recommended sensors: DHT22 (temperature & humidity), MQ-135 (air quality / CO2), HC-SR501 PIR (motion), and BMP280 (barometric pressure). Readings are transmitted every 5 seconds over USB serial to the Raspberry Pi. If using an ESP32 module, data is sent via WiFi as HTTP POST requests or MQTT messages to the Pi gateway.

Stage 2 -- Edge Gateway & Pre-Processing (Raspberry Pi)

The Raspberry Pi runs a lightweight Python gateway script that:

- Reads Arduino serial output using the *pyserial* library.
- Injects ISO 8601 timestamps and sensor node IDs into each reading.
- Filters out-of-range outliers before forwarding.
- Batches readings into CSV files (every 100 records or every 60 seconds).
- Uploads batched files to HDFS via the WebHDFS REST API (HTTP PUT to NameNode port 50070).

Sample Python gateway snippet for the Raspberry Pi:

```
import serial, requests, datetime

ser = serial.Serial('/dev/ttyACM0', 9600)
HDFS = 'http://10.0.10.10:50070/webhdfs/v1'
buf = []

while True:
    line = ser.readline().decode().strip() # e.g. '24.5,62.1,420'
    temp, hum, co2 = line.split(',')
    record = {
        'ts': datetime.datetime.utcnow().isoformat(),
        'node': 'pi-gateway-01',
        'temp': float(temp), 'hum': float(hum), 'co2': int(co2)
    }
```

```

buf.append(record)
if len(buf) >= 100:
    csv_data = to_csv(buf) # convert list to CSV bytes
    path = f"/iot/sensors/{record['ts'][:10]}/batch.csv"
    requests.put(f'{HDFS}{path}?op=CREATE&overwrite;=true',
    data=csv_data)
buf.clear()

```

Stage 3 -- Distributed Storage & Processing (Hadoop)

HDFS stores raw sensor batches at /iot/sensors/YYYY-MM-DD/. An Apache Hive external table is created over this directory, enabling standard SQL analytics against the live sensor data:

```

CREATE EXTERNAL TABLE sensor_readings (
    ts STRING,
    node STRING,
    temp FLOAT,
    humidity FLOAT,
    co2 INT
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/iot/sensors/';

```

Stage 4 -- Real-Time Visualisation (Django Dashboard)

The Django application queries the Hive table via PyHive and renders results as interactive charts. A dashboard page auto-refreshes every 30 seconds, displaying live temperature and humidity readings per node, 24-hour CO2 trend graphs, alert banners when thresholds are exceeded, and historical heatmaps of sensor distribution over time.

Network Integration of IoT Devices

Raspberry Pi gateways are placed on VLAN 40 (10.0.40.0/24), isolated from the Hadoop cluster and Django tiers. A specific ACL permits only the necessary WebHDFS and Kafka traffic from the IoT VLAN to the Hadoop VLAN:

```

ip access-list extended IOT_TO_HADOOP
permit tcp 10.0.40.0 0.0.0.255 10.0.10.0 0.0.0.255 eq 50070
permit tcp 10.0.40.0 0.0.0.255 10.0.10.0 0.0.0.255 eq 9092
deny ip any any
interface Vlan40
ip access-group IOT_TO_HADOOP in

```

3. Routing Configuration

3.1 Router Configuration

Routers are essential for interconnecting the different VLANs and providing external network access. Basic router configuration involves setting up hostnames, securing console and enable access, and configuring interfaces. The router acts as the default gateway for traffic leaving each VLAN and as the primary enforcement point for inter-zone security policies.

3.2 Static and Dynamic Routing

Both static and dynamic routing will be employed to ensure efficient and resilient traffic flow within the network.

Static Routing: Used for specific, predictable routes, such as directing traffic from the Django application servers to the HiveServer2 instance, from Raspberry Pi gateways to the NameNode, or for default routes to the internet gateway. This ensures critical internal communication paths are well-defined.

Dynamic Routing (OSPF/EIGRP): For larger, more complex deployments or environments where network topology changes frequently, dynamic routing protocols like OSPF (Open Shortest Path First) or EIGRP (Enhanced Interior Gateway Routing Protocol) can be used. These protocols dynamically adjust to network changes, providing fault tolerance and load balancing across multiple paths.

4. Switching Configuration

4.1 Switch Configuration

Switches are configured to create VLANs, manage port assignments, and ensure efficient data forwarding within each network segment. Basic switch configuration includes hostname, security, and interface settings. Trunk ports are configured between switches and the Layer 3 switch to carry traffic from all VLANs simultaneously.

4.2 VLANs

Virtual LANs (VLANs) are crucial for segmenting the network, enhancing security, and improving performance by reducing broadcast domains. For this project, five distinct VLANs will be created: Hadoop Cluster (VLAN 10), Django Application (VLAN 20), Management (VLAN 30), IoT Edge (VLAN 40), and Client Access (VLAN 50). This five-VLAN design ensures that IoT sensor traffic is isolated from critical Hadoop and Django components.

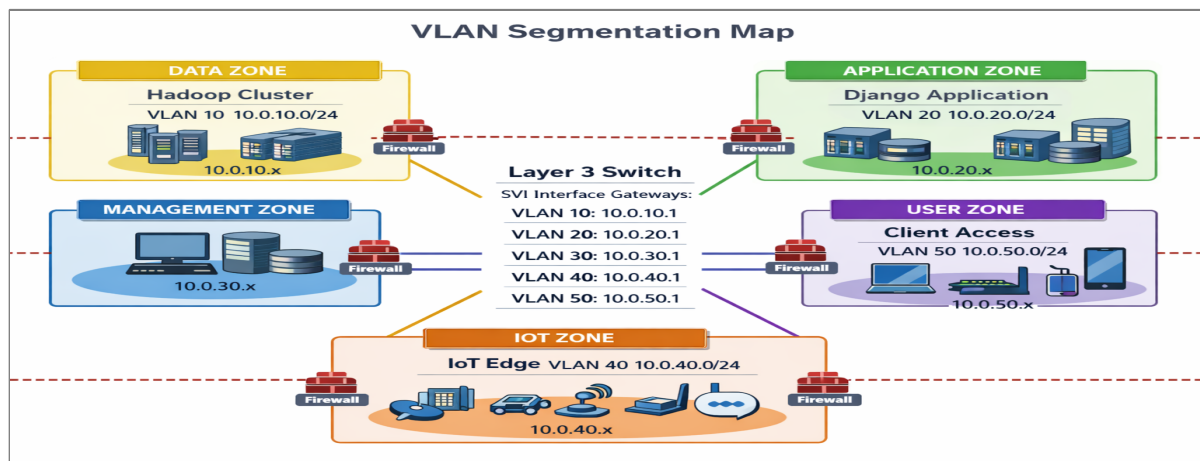


Figure 5: VLAN Segmentation and Network Zones

5. Inter-VLAN Routing

5.1 Layer 3 switching using SVIs

Inter-VLAN routing will be primarily handled by Layer 3 switches using Switched Virtual Interfaces (SVIs). This allows for efficient routing between VLANs without requiring traffic to traverse a dedicated router, improving performance and reducing latency. Each VLAN is assigned an SVI with the subnet default gateway IP address.

5.2 Subnetting

Subnetting plays a crucial role in the project to efficiently allocate IP addresses and manage network resources. The base network address of 10.0.0.0/16 is subnetted to accommodate different tiers. For example, VLAN 10 uses the subnet 10.0.10.0/24, while VLAN 40 (IoT Edge) uses 10.0.40.0/24. Subnetting ensures that each VLAN has its own distinct range of IP addresses, preventing overlap and facilitating organised addressing within the network. This approach enhances security, simplifies network management, and supports future scalability by providing a structured allocation of IP resources to individual VLANs.

6. Security Measures

6.1 Access Control Lists (ACLs)

Access Control Lists (ACLs) will be implemented on routers and Layer 3 switches to filter network traffic based on defined criteria, such as source/destination IP addresses, ports, and protocols. This is crucial for restricting unauthorised access to Hadoop components, the Django application, and the IoT edge layer.

Example ACL for Django to HiveServer2 Communication:

```
ip access-list extended DJANGO_TO_HIVE
permit tcp 10.0.20.0 0.0.0.255 host 10.0.13.X eq 10000
deny ip any any
exit
interface Vlan20
ip access-group DJANGO_TO_HIVE in
exit
```

(Where 10.0.13.X is the IP of the HiveServer2 and 10000 is the default HiveServer2 port)

Example ACL for Management Access:

```
ip access-list extended MANAGEMENT_ACCESS
permit tcp host [Admin_Workstation_IP] any eq 22
permit tcp host [Admin_Workstation_IP] any eq 80
permit tcp host [Admin_Workstation_IP] any eq 443
deny ip any any
exit
interface Vlan30
ip access-group MANAGEMENT_ACCESS in
exit
```

Example ACL for IoT Edge to Hadoop (permits only WebHDFS and Kafka):

```
ip access-list extended IOT_TO_HADOOP
permit tcp 10.0.40.0 0.0.0.255 10.0.10.0 0.0.0.255 eq 50070
permit tcp 10.0.40.0 0.0.0.255 10.0.10.0 0.0.0.255 eq 9092
deny ip any any
exit
interface Vlan40
ip access-group IOT_TO_HADOOP in
exit
```

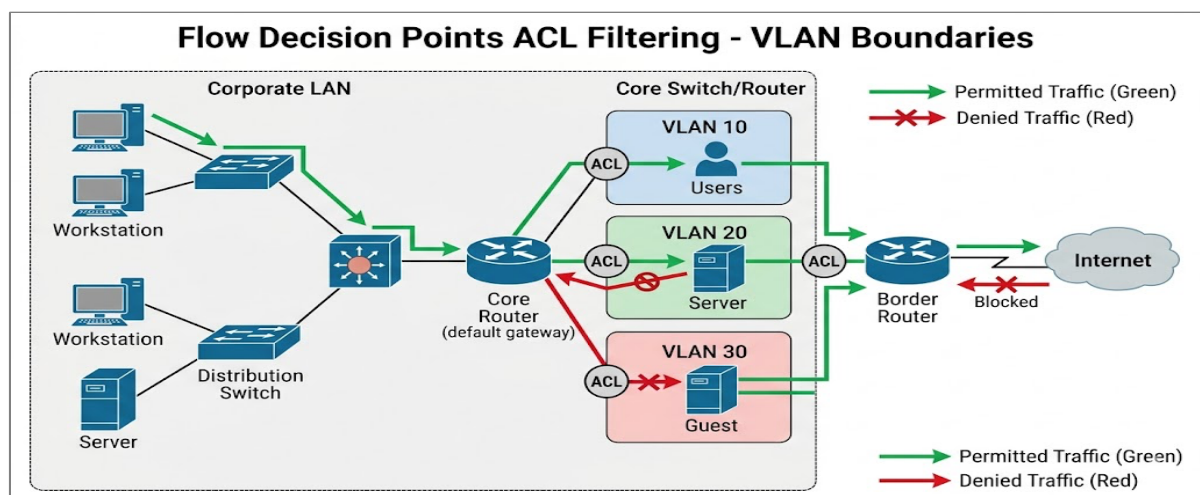


Figure 6: ACL Traffic Filtering Flow

6.2 NAT and PAT

Network Address Translation (NAT) and Port Address Translation (PAT) will be used primarily at the network edge (router connecting to the internet) to allow internal Django application servers to communicate with external services (if needed) and to protect internal IP addressing schemes from public exposure.

Example NAT/PAT Configuration:

```
ip nat inside source list 1 interface GigabitEthernet0/0 overload
access-list 1 permit 10.0.20.0 0.0.0.255
interface Vlan20
ip nat inside
interface GigabitEthernet0/0
ip nat outside
```

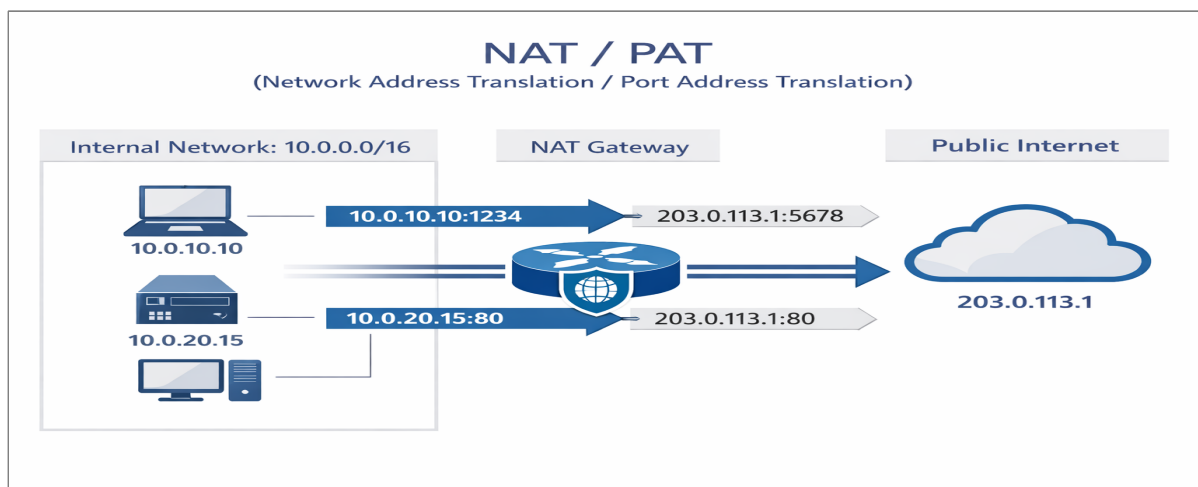


Figure 7: NAT/PAT Address Translation

6.3 Port Security

Port security will be implemented on Layer 2 access switches connected to individual Hadoop nodes, Django servers, and Raspberry Pi gateways. This limits the number of MAC addresses allowed on a specific switch port, preventing unauthorised devices from connecting to critical infrastructure.

Example Port Security Configuration:

```
interface FastEthernet0/1
switchport mode access
switchport access vlan 10
switchport port-security
switchport port-security maximum 1
switchport port-security mac-address sticky
switchport port-security violation shutdown
```

This configuration ensures that only one device with a learned MAC address can connect to the port, and any violation will shut down the port, alerting administrators to potential security breaches.

7. Quality of Service (QoS)

7.1 QoS Configuration

Quality of Service (QoS) will be configured to prioritise critical network traffic, ensuring that the Big Data Analytics Dashboard remains responsive and that essential Hadoop operations are not hampered by less critical traffic. While a full QoS implementation can be complex, key areas of focus include:

- Prioritising HiveServer2 Traffic: Ensuring that queries from the Django application to HiveServer2 receive preferential treatment.
- Prioritising HDFS Data Transfer: For large data ingestions or movements within the Hadoop cluster, ensuring HDFS traffic has adequate bandwidth.
- Prioritising IoT Sensor Ingestion: Ensuring steady upload bandwidth from Raspberry Pi gateways to HDFS (WebHDFS port 50070).
- Prioritising Django Web Traffic: Ensuring that user interactions with the dashboard are smooth and responsive.

Example QoS Policy (Conceptual):

```
class-map match-all HIVE_TRAFFIC
match port tcp 10000
class-map match-all HDFS_TRAFFIC
match port tcp 9000
class-map match-all IOT_TRAFFIC
match port tcp 50070
policy-map BIG_DATA_QOS
class HIVE_TRAFFIC
priority percent 40
class HDFS_TRAFFIC
bandwidth percent 30
class IOT_TRAFFIC
bandwidth percent 15
class class-default
fair-queue
interface GigabitEthernet0/1
service-policy output BIG_DATA_QOS
```

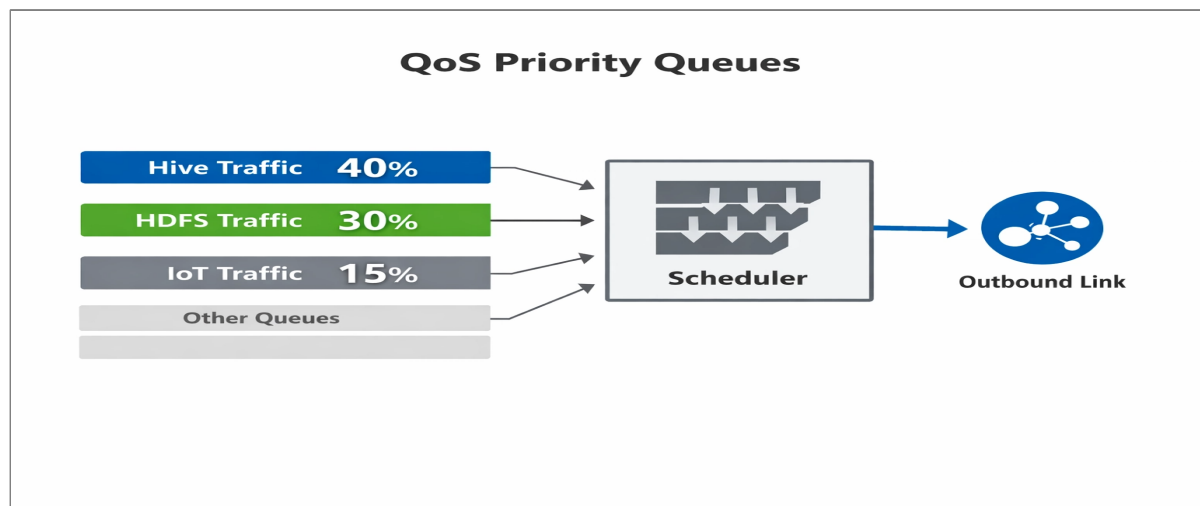


Figure 8: QoS Traffic Priority Queues

8. Monitoring and Management

8.1 SNMP Configuration

Simple Network Management Protocol (SNMP) will be configured on all network devices (routers, switches) and potentially on Hadoop nodes (via agents) to facilitate centralised monitoring and management. This allows network administrators to collect device statistics, monitor performance, and receive alerts.

```
snmp-server community public RO
snmp-server enable traps
snmp-server host 10.0.30.10 public
```

(Where 10.0.30.10 is the IP address of a dedicated monitoring server in the Management VLAN)

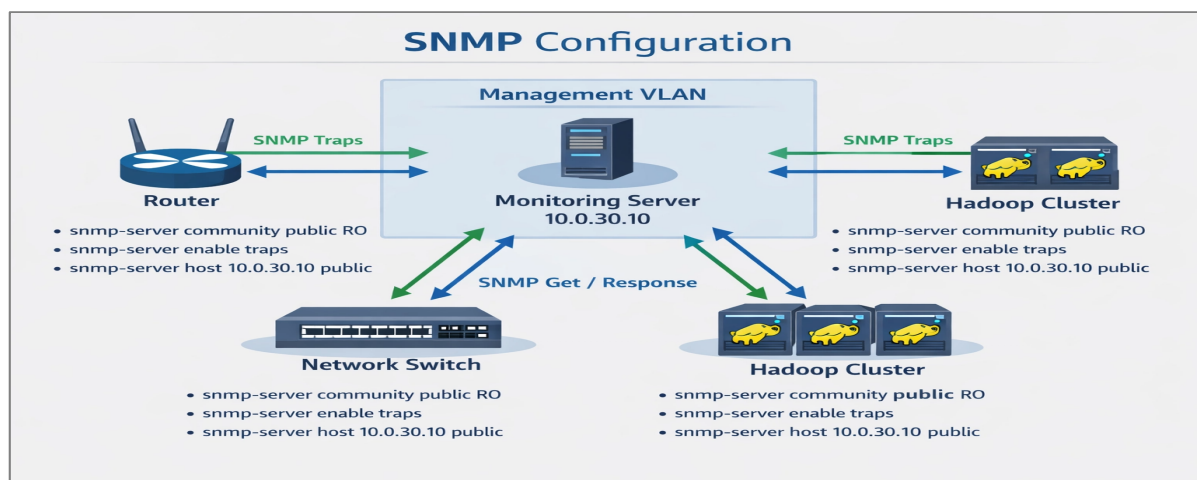


Figure 9: SNMP Monitoring Architecture

8.2 Logging and Alerts

Comprehensive logging and alerting mechanisms are essential for maintaining the health and security of the Big Data Analytics Dashboard. Logs from network devices, Hadoop components, the Django application, and IoT edge gateways will be aggregated and analysed.

Network Device Logging:

```
logging buffered informational
logging 10.0.30.10
snmp-server enable traps syslog
```

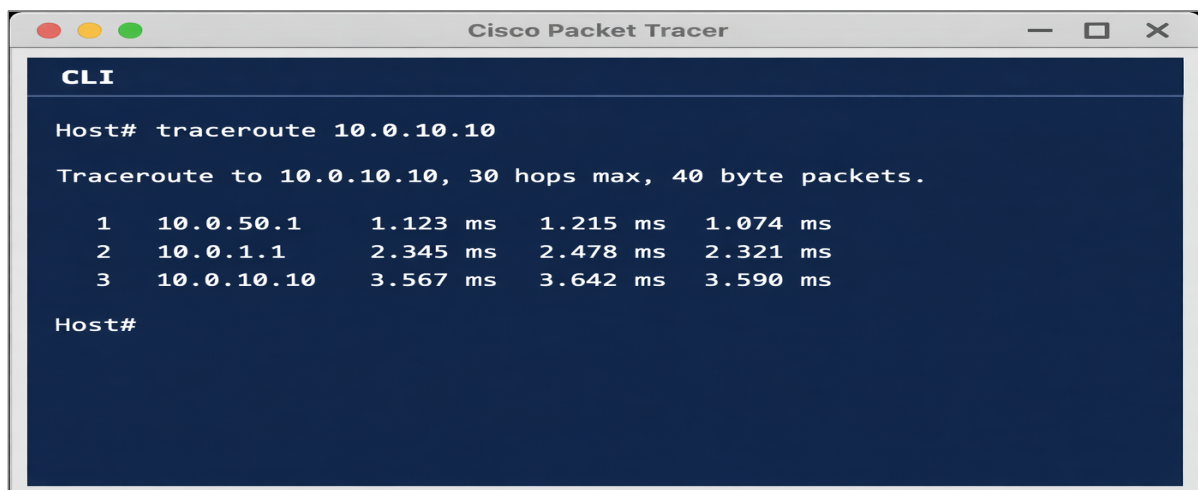
- **Hadoop Logging:** Hadoop components generate extensive logs (e.g., NameNode logs, DataNode logs, YARN logs). These logs will be configured to be stored in a centralised logging system (e.g., ELK stack – Elasticsearch, Logstash, Kibana) for analysis and alerting.
- **Django Application Logging:** The Django application will be configured to log errors, warnings, and access attempts. These logs will also be directed to the centralised logging system.
- **IoT Edge Logging:** Raspberry Pi gateway scripts log upload success/failure events and sensor anomalies to the centralised system for visibility.

9. Testing and Validation

9.1 Simulation

While a full-scale simulation of a Hadoop cluster and Django application within a network simulator like Cisco Packet Tracer is challenging, network design aspects can be simulated. The focus of simulation would be on:

- Network Topology Verification: Ensuring that the designed network topology (VLANs, routing paths) is correctly implemented and that all components can communicate as intended.
- Connectivity Testing: Verifying end-to-end connectivity between Django servers and HiveServer2, between Hadoop nodes, and from Raspberry Pi gateways to the NameNode WebHDFS endpoint.
- ACL Validation: Testing ACLs to ensure they correctly permit authorised traffic and deny unauthorised access, including the IoT_TO_HADOOP ACL.
- Routing Path Verification: Using tools like traceroute to confirm that data packets follow the expected paths.

A screenshot of the Cisco Packet Tracer application window. The title bar reads "Cisco Packet Tracer" with standard window controls. The main area is a dark blue terminal window titled "CLI". It shows the command "Host# traceroute 10.0.10.10" being entered. Below the command, it says "Traceroute to 10.0.10.10, 30 hops max, 40 byte packets." followed by a table of results for three hops. The first hop is 10.0.50.1 with times 1.123 ms, 1.215 ms, and 1.074 ms. The second hop is 10.0.1.1 with times 2.345 ms, 2.478 ms, and 2.321 ms. The third hop is 10.0.10.10 with times 3.567 ms, 3.642 ms, and 3.590 ms. The prompt "Host#" is at the bottom.

```
CLI
Host# traceroute 10.0.10.10

Traceroute to 10.0.10.10, 30 hops max, 40 byte packets.

 1  10.0.50.1      1.123 ms   1.215 ms   1.074 ms
 2  10.0.1.1       2.345 ms   2.478 ms   2.321 ms
 3  10.0.10.10    3.567 ms   3.642 ms   3.590 ms

Host#
```

Figure 10: Traceroute Successful

9.2 Troubleshooting

During the testing phase, various troubleshooting steps will be employed to identify and resolve issues:

- Network Connectivity Issues: Using ping, traceroute, and show ip interface brief commands on network devices to diagnose connectivity problems between different tiers and components.
- Hadoop Component Health: Checking Hadoop daemon logs, jps command on nodes, and Hadoop web UIs (e.g., NameNode UI port 9870, ResourceManager UI port 8088) to ensure all services are running correctly.
- Django Application Errors: Reviewing Django logs, web server logs (e.g., Nginx/Apache), and browser developer tools to identify application-level issues or communication failures with the Hadoop cluster.
- Raspberry Pi Gateway Issues: Verifying WebHDFS connectivity using curl to NameNode port 50070, checking Python gateway script logs, and confirming IoT VLAN ACL rules are correctly applied.
- Hive Query Failures: Examining HiveServer2 logs and the output of beeline or other Hive clients to troubleshoot query execution problems.
- Security Policy Enforcement: Verifying that ACLs and port security configurations are effectively blocking unauthorised traffic and connections.

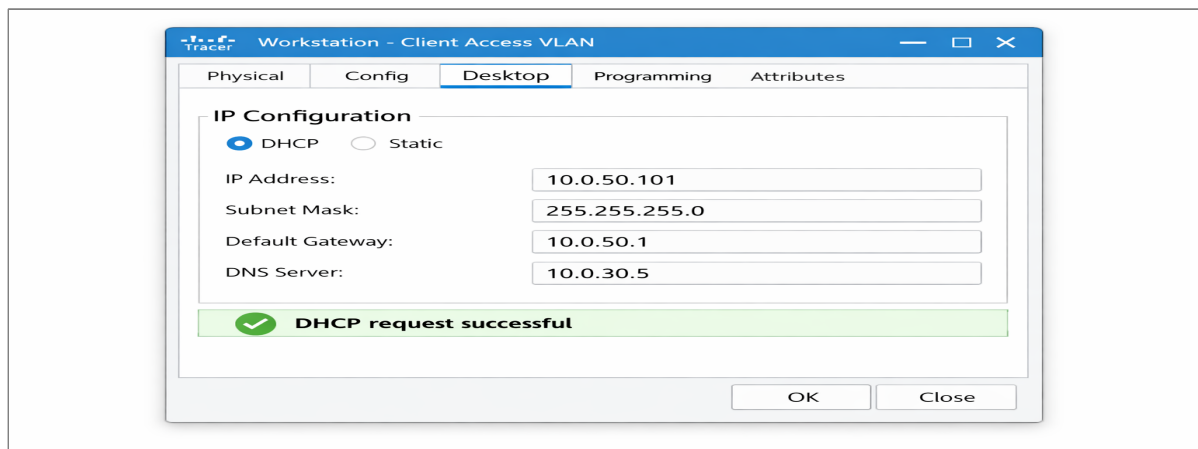


Figure 11: DHCP IP Allocation

10. Results and Evaluation

10.1 Performance Metrics

Performance metrics will be collected and evaluated to assess the efficiency, scalability, and responsiveness of the Big Data Analytics Dashboard. Key metrics include:

Hadoop Cluster Performance:

- HDFS Throughput: Data read/write speeds to and from HDFS.
- YARN Resource Utilisation: CPU and memory usage across the cluster.
- Hive Query Latency: Time taken to execute queries from Django through HiveServer2.
- MapReduce/Spark Job Completion Times: For any batch processing jobs.

Django Application Performance:

- Response Time: Time taken for the dashboard to load and display data after a user request.
- Concurrent Users: Number of users the application can handle simultaneously without performance degradation.
- API Latency: Response times for API calls made by the Django front-end to the backend.

Network Performance:

- Bandwidth Utilisation: Monitoring traffic on critical links between tiers.
- Network Latency: Ping times between Django servers and Hadoop nodes.
- Packet Loss: Ensuring minimal packet loss across the network.
- Security Audit Results: Regular security audits and penetration testing results to evaluate the effectiveness of implemented security measures.

Visualisations (e.g., graphs of query latency over time, HDFS throughput, CPU utilisation) will be used to present these performance metrics effectively.

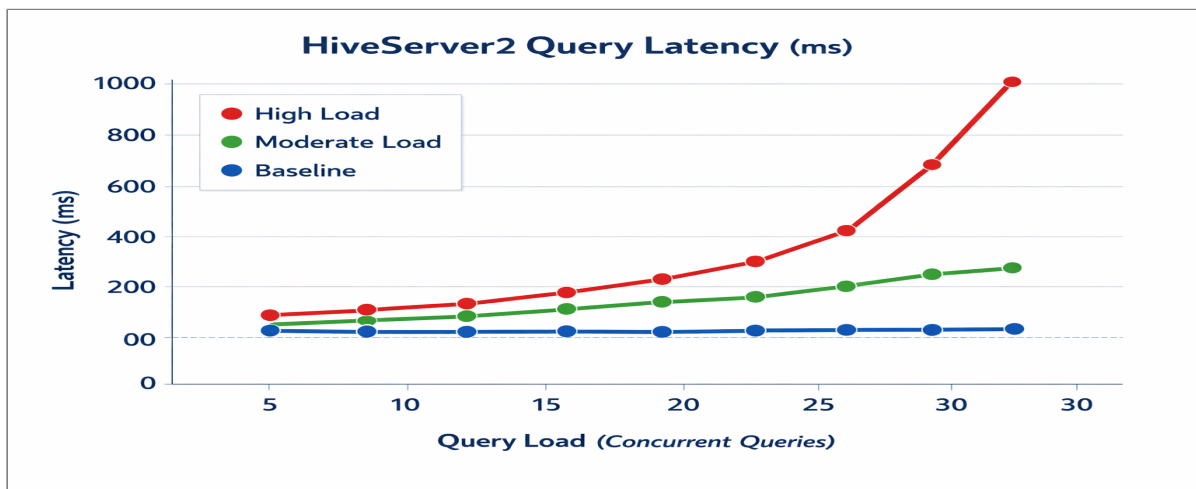


Figure 12: HiveServer2 Query Latency (ms) vs. Query Load

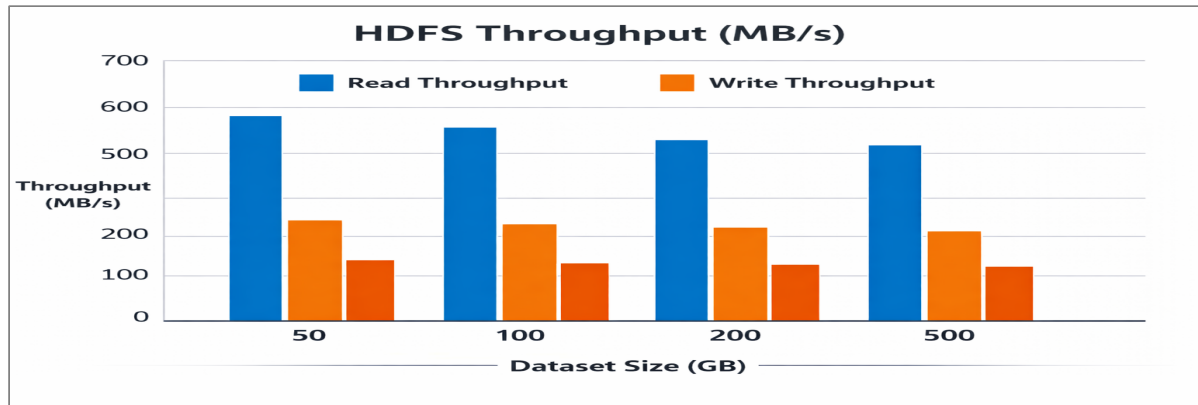


Figure 13: HDFS Throughput (MB/s) vs. Dataset Size

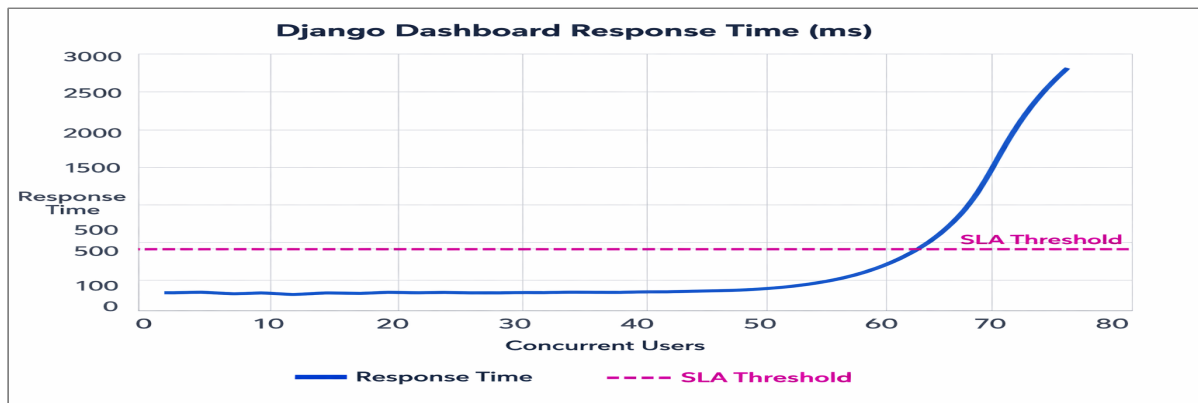


Figure 14: Django Dashboard Response Time vs. Concurrent Users

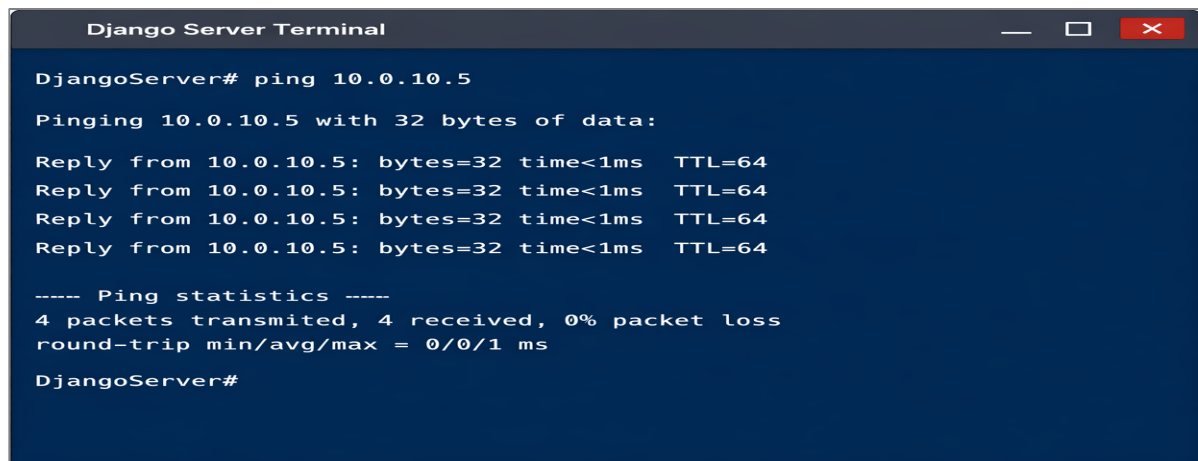


Figure 15: Performance Measure Through Ping Time

11. Real-World Applications

The Big Data Analytics Dashboard architecture presented in this report has direct and significant applicability across multiple high-impact industries. The following scenarios demonstrate how the specific technical components of this project map to real deployments.

11.1 Smart Campus / Building Monitoring

Arduino sensors and Raspberry Pi gateways monitor temperature, CO2, occupancy, and energy consumption across classrooms, labs, and server rooms. The Hadoop and Django platform provides facilities management with a live view of campus conditions, energy waste alerts, and historical trend analysis. This scenario is immediately deployable at an institution such as SRM IST. HDFS stores time-series sensor readings from every room; Hive queries compute average CO2 per classroom per day, flagging poorly ventilated spaces; and the Django dashboard renders a campus map with colour-coded room status. The VLAN and ACL architecture maps directly to campus network security requirements.

11.2 Healthcare & Hospital Data Analytics

Hospitals generate terabytes of patient records, medical imaging, lab results, and IoT-based vital sign monitoring data daily. The Hadoop and Django architecture can centralise and process this data for clinical decision support, with Arduino and Raspberry Pi nodes acting as bedside sensor aggregators feeding patient vitals into HDFS. The VLAN segmentation described in Section 4.2 maps directly to HIPAA-compliant network isolation requirements, and the ACLs in Section 6.1 enforce role-based data access, ensuring only authorised clinical systems reach patient data nodes.

11.3 Financial Services & Fraud Detection

Banks and fintech companies process millions of transactions per hour. Real-time fraud detection requires processing streams of transactional data at scale, which is precisely Hadoop's design purpose. The QoS configuration in Section 7 ensures fraud-detection queries are prioritised over routine reporting traffic, and the port security measures in Section 6.3 map directly to PCI-DSS requirements for securing financial data infrastructure. The Django dashboard serves compliance and fraud analytics teams with query-driven reports.

11.4 Telecommunications Network Analytics

Telecom providers generate enormous volumes of Call Detail Records (CDRs) and network performance data. Hadoop is the industry-standard platform for telecom data processing. CDRs and network KPIs such as latency, throughput, and packet loss (directly aligned with Section 10.1) are stored in HDFS, processed by MapReduce jobs, and surfaced on the Django dashboard for Network Operations Centre (NOC) engineers. The SNMP monitoring architecture described in Section 8.1 is the exact protocol used in commercial telecom NOCs. The OSPF/EIGRP dynamic routing in Section 3.2 mirrors protocols used in telecom backbone networks.

11.5 Agricultural IoT & Precision Farming

Arduino soil moisture, temperature, and humidity sensors distributed across agricultural fields feed data via Raspberry Pi cellular gateways into the central Hadoop cluster. Farmers and agronomists access trend data and irrigation recommendations through the Django dashboard. Hive queries identify optimal irrigation schedules based on historical moisture and temperature patterns. The IoT data ingestion pipeline described in Section 2.4 is directly applicable with minor sensor hardware substitutions.

11.6 E-Commerce & Retail Analytics

E-commerce platforms process billions of user clicks, product views, and purchase events. Hadoop enables batch processing of this data to power recommendation engines and inventory forecasting, while Django serves as the business intelligence dashboard for management teams. The load balancers described in Section 2.2 handle traffic spikes during major sales events, and the YARN resource management configuration ensures that overnight MapReduce jobs for customer segmentation do not impact daytime dashboard responsiveness.

12. Conclusion

12.1 Summary

This case study has outlined the design and proposed implementation of a Big Data Analytics Dashboard, integrating Apache Hadoop for scalable data processing and storage with Django for an interactive web interface, and extended the architecture with a practical IoT data ingestion layer using Arduino microcontrollers and Raspberry Pi edge gateways. The report detailed a robust network architecture, including five-VLAN segmentation, IP addressing, and routing configurations, specifically tailored to support the distributed nature of Hadoop, the accessibility requirements of a web application, and the data ingestion needs of IoT edge devices. Comprehensive security measures, such as ACLs, NAT/PAT, and port security, were discussed to safeguard the system across all tiers. Furthermore, the importance of QoS for prioritising critical traffic and extensive monitoring and logging for operational visibility were highlighted. Real-world applications across six industry domains confirm the practical relevance of this architecture beyond an academic context. The proposed testing and validation strategies aim to ensure the system's functionality, performance, and reliability.

12.2 Lessons Learned

Throughout the conceptualisation and design of this integrated system, several key lessons have been reinforced:

- **Integrated Design is Paramount:** Successful big data solutions require a holistic approach, where network design, security, IoT data collection, and application development are tightly integrated from the outset.
- **IoT Adds Real-World Credibility:** Connecting physical Arduino sensors to the Hadoop pipeline transforms the project from a simulation exercise into a deployable, live data system that demonstrates the full stack from hardware to web.
- **Scalability and Redundancy are Non-Negotiable:** For big data environments, designing for scalability and building in redundancy at every layer (network, compute, storage, edge) is crucial for continuous operation and future growth.
- **Security by Design:** Implementing security measures like ACLs and port security at the network level, alongside application-level security, is fundamental to protecting sensitive data across all tiers including the IoT edge.
- **Proactive Monitoring:** A robust monitoring and logging strategy is vital for early detection of issues, performance bottlenecks, and security incidents in complex distributed systems.
- **Interoperability Challenges:** Integrating diverse technologies like Hadoop, Django, Arduino, and Raspberry Pi requires careful consideration of communication protocols, APIs (e.g., PyHive for HiveServer2, WebHDFS for HDFS), and data formats.
- **Performance Optimisation:** Continuous evaluation of performance metrics and fine-tuning of both network and application configurations are necessary to maintain optimal system responsiveness.

13. References

- [1] Apache Hadoop. (n.d.). Apache Hadoop. Retrieved from <https://hadoop.apache.org/>
- [2] Django Project. (n.d.). The Web framework for perfectionists with deadlines. Retrieved from <https://www.djangoproject.com/>
- [3] PyHive. (n.d.). PyHive: Python interface to Hive and Presto. Retrieved from <https://github.com/dropbox/PyHive>
- [4] Apache Hive. (n.d.). Apache Hive. Retrieved from <https://hive.apache.org/>
- [5] Cisco. (n.d.). Cisco IOS Command Reference. Retrieved from <https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/fundamentals/command/cf-cr-xe.html>
- [6] Raspberry Pi Foundation. (n.d.). Raspberry Pi Documentation. Retrieved from <https://www.raspberrypi.com/documentation/>
- [7] Arduino. (n.d.). Arduino Reference and Documentation. Retrieved from <https://www.arduino.cc/reference/>
- [8] Apache Software Foundation. (n.d.). WebHDFS REST API. Retrieved from <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/WebHDFS.html>
- [9] Elastic. (n.d.). ELK Stack: Elasticsearch, Logstash, Kibana. Retrieved from <https://www.elastic.co/what-is/elk-stack>
- [10] Apache Kafka. (n.d.). Apache Kafka Documentation. Retrieved from <https://kafka.apache.org/documentation/>

14. Appendices

Abbreviations:

- ACL – Access Control List
- DHCP – Dynamic Host Configuration Protocol
- HDFS – Hadoop Distributed File System
- IP – Internet Protocol
- IoT – Internet of Things
- ISP – Internet Service Provider
- LAN – Local Area Network
- MAC – Media Access Control
- MQTT – Message Queuing Telemetry Transport
- NAT – Network Address Translation
- OSPF – Open Shortest Path First
- PAT – Port Address Translation
- QoS – Quality of Service
- RPi – Raspberry Pi
- SNMP – Simple Network Management Protocol
- SSH – Secure Shell
- SVI – Switched Virtual Interface
- VLAN – Virtual Local Area Network
- YARN – Yet Another Resource Negotiator